Revisit to CPE233 (Digital Design)

Authors:
WhisperingRock

Professor:

Class
Quarter
July 20, 2026

Contents

Note : All assignments are in the **CPE 233 Lab Manual**.

0.1 Hardware

0.1.1 HW1 - Program ROM

Reverse Engineer'd Table

```
0x0000_0000: 11000537
0x0000_0004: 00a00f13
0x0000_0008: 00051783
0x0000_000C: 41e7da33
0x0000_0010: 00fa4633
0x0000_0014: 04c52023
0x0000_0018: ff1ff06f
```

Figure 1: Which Table? This table

RISC-V Instruction Set Summary

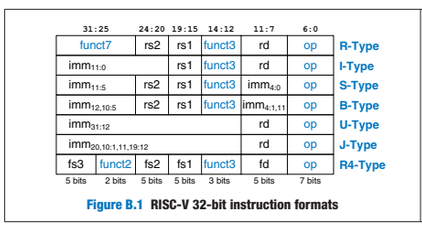

Figure B.1 RISC-V 32-bit instruction formats

- imm: signed immediate in $\mathbf{imm}_{11:0}$
- uimm: 5-bit unsigned immediate in $\mathbf{imm}_{4:0}$
- upimm: 20 upper bits of a 32-bit immediate, in $\mathbf{imm}_{31:12}$
- Address: memory address: **rs1** + SignExt($\mathbf{imm}_{11:0}$)
- [Address]: data at memory location Address
- BTA: branch target address: PC + SignExt({$\mathbf{imm}_{12:1}$, 1'b0})
- JTA: jump target address: PC + SignExt({$\mathbf{imm}_{20:1}$, 1'b0})
- label: text indicating instruction address
- SignExt: value sign-extended to 32 bits
- ZeroExt: value zero-extended to 32 bits
- csr: control and status register

Table B.1 RV32I: RISC-V integer instructions

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	–	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address]$_{7:0}$)
0000011 (3)	001	–	I	lh rd, imm(rs1)	load half	rd = SignExt([Address]$_{15:0}$)
0000011 (3)	010	–	I	lw rd, imm(rs1)	load word	rd = [Address]$_{31:0}$
0000011 (3)	100	–	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address]$_{7:0}$)
0000011 (3)	101	–	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address]$_{15:0}$)
0010011 (19)	000	–	I	addi rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	slli rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm
0010011 (19)	010	–	I	slti rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	–	I	sltiu rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	–	I	xori rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	srli rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm
0010011 (19)	101	0100000*	I	srai rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm
0010011 (19)	110	–	I	ori rd, rs1, imm	or immediate	rd = rs1 | SignExt(imm)
0010011 (19)	111	–	I	andi rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	–	–	U	auipc rd, upimm	add upper immediate to PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	–	S	sb rs2, imm(rs1)	store byte	[Address]$_{7:0}$ = rs2$_{7:0}$
0100011 (35)	001	–	S	sh rs2, imm(rs1)	store half	[Address]$_{15:0}$ = rs2$_{15:0}$
0100011 (35)	010	–	S	sw rs2, imm(rs1)	store word	[Address]$_{31:0}$ = rs2
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 − rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << rs2$_{4:0}$
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl rd, rs1, rs2	shift right logical	rd = rs1 >> rs2$_{4:0}$
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2$_{4:0}$
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 | rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2
0110111 (55)	–	–	U	lui rd, upimm	load upper immediate	rd = {upimm, 12'b0}
1100011 (99)	000	–	B	beq rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	–	B	bne rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	–	B	blt rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	–	B	bge rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	–	B	bltu rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	–	B	bgeu rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	–	I	jalr rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	–	–	J	jal rd, label	jump and link	PC = JTA, rd = PC + 4

*Encoded in instr$_{31:25}$, the upper seven bits of the immediate field

ProgROM Addr	Machine Code	Type	Assembly Instruction
0000	0x1100 0537		
	0001 0001 0000 0000 0000 0101 0 011 0111	(55) U	lui rd, upimm
	0001 0001 0000 0000 0000 0101 0 011 0111	(55) U	lui x10, 0x11000
0004	0x00A0 0F13		
	0000 0000 1010 0000 0 000 1111 0 001 0011	(19)I	addi rd, rs1, imm
	0000 0000 1010 0000 0 000 1111 0 001 0011	(19)I	addi x30, x0, 0x00A
0008	0x0005 1783		
	0000 0000 0000 0101 0 001 0111 1 000 0011	(3)I	lh rd, imm(rs1)
	0000 0000 0000 0101 0 001 0111 1 000 0011	(3)I	lh x15, 0(x10)
000C	0x41E7 DA33		
	0100 000 1 1110 0111 1 101 1010 0 011 0011	(51)R	sra rd, rs1, rs2
	0100 000 1 1110 0111 1 101 1010 0 011 0011	(51)R	sra x20, x15, x30
0010	0x00FA 4633		
	0000 0000 1111 1010 0 100 0110 0 011 0011	(51)R	xor rd, rs1, rs2
	0000 000 0 1111 1010 0 100 0110 0 011 0011	(51)R	xor x12, x20, x15
0014	0x04C5 2023		
	0000 0100 1100 0101 0 010 0000 0 010 0011	(35)S	sw rs2, imm(rs1)
	0000 010 0 1100 0101 0 010 0000 0 010 0011	(35)S	sw x12, 0x040(x10)
0018	0xFF1F F06F		
	1111 1111 0001 1111 1111 0000 0 110 1111	(111)J	jal rd, label
	1111 1111 0001 1111 1111 0000 0 110 1111	(111)J	jal x0, -16

Table 1: Reverse Engineering *otter_memory.mem* segment

For the last command, some explanation is required here. Consider

- $Imm[20] = B31 = 1$
- $Imm[10:1] = B30:B21 = 1111111000$
- $Imm[11] = B20 = 1$
- $Imm[19:12] = B19:B12 = 11111111$
- $Imm[0]$ is always zero
- Therefore, $Imm = 0x1FFFF0$

To find the decimal representation, consider adding our large negative number to the next highest max.

$$\begin{aligned} 2^{21} - 0x1FFFF0 &= 0x200000 - 0x1FFFF0 \\ &= -0x10 \\ &= -16 \end{aligned}$$

BUT WAIT! `jal` doesn't take immediates, only labels.

True, our label must be 4 spaces prior to this jump - fill this portion in with a label in the asm and respect the spacing.

Assembly Code

Listing 1: Assembly of HW1

```
# Author : WhisperingRock
#
# Purpose : Confirm decoded machine code (by hand)
#     functionality matches what the RARS compiler
#     produces.



.data
A:  .word 0x1234    # dummy value to see change

.global _start

.text
.equ  MMIO, 0x11000000

_start:
  # ~~~~ init registers ~~~~
  li t1, MMIO
  lw t2, A
  sw t2, 0(t1)

  # ~~~~ decoded instr ~~~~
```

```
  #lui x10, 0x11000
  #addi x30, x0, 0x00A
#loop:
  #lh x15, 0(x10)
  #sra x20, x15, x30
  #xor x12, x20, x15
  #sw x12, 0x040(x10)
  #jal x0, loop   #loop label is 4 spaces back

  lui a0, 0x11000       # a0 = x10
  addi t5, zero, 0x00A      # zero = x0, t5 = x30
loop:
  lh a5, 0(a0)          # a5 = x15
  sra s4, a5, t5          # s4 = x20
  xor a2, s4, a5          # a2 = x12
  sw a2, 64(a0)
  jal zero, loop

  # ~~~~ syscall exit status ~~~~
  li a7, 10    # load syscall call num for exit()
  li a0, 0     # EXIT_SUCCESS code
  ecall      # make the system call
```

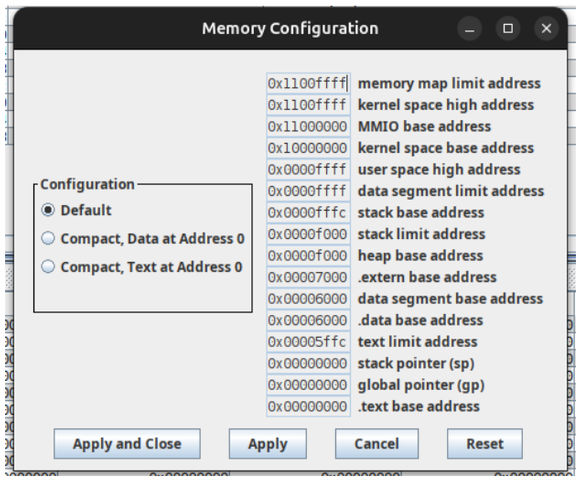

Figure 2: Segment Address Config

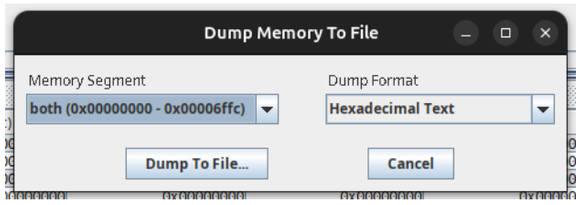

Figure 3: Memory Dump Config

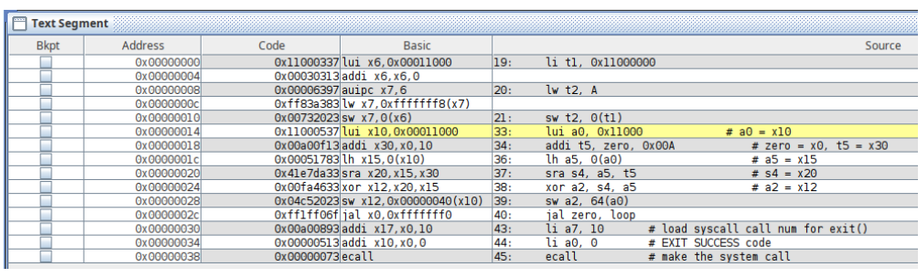

Figure 4: RARS instruction addresses alongside code

Vivado Simulation

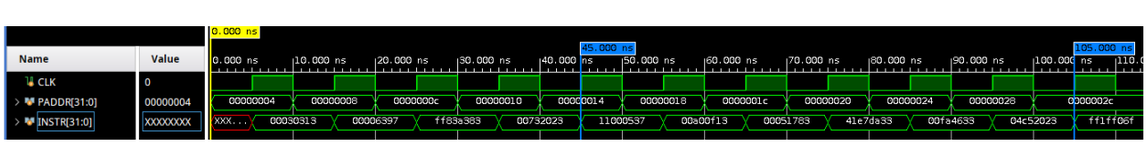

Figure 5: Simulation waveform of `ProgROM_tb.sv` running `otter_memory_hw1.mem`

By inspection, the instructions in the waveform 5 match those in RARS4 and our decoded table 1

0.1.2 HW2 - Program Counter

Intended Behavior(s)

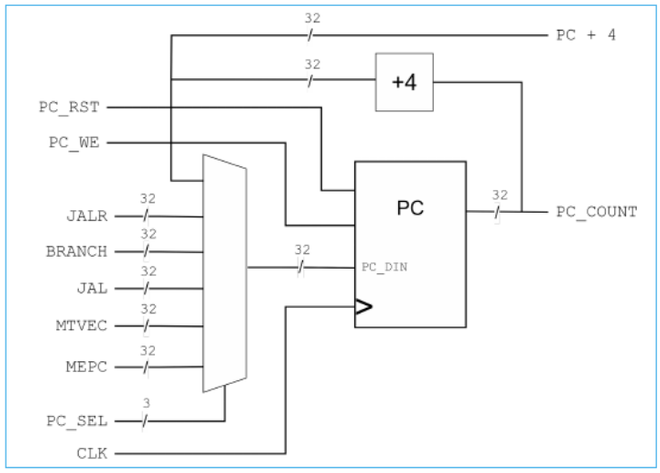

Figure 6: Program Counter Test Block Design

Structural Design(s)

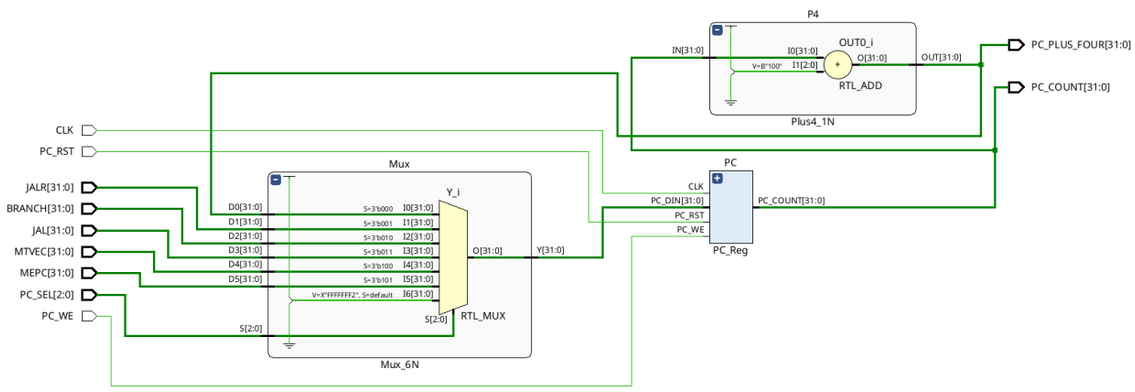

Figure 7: RTL Design of `PC_TestBlock.sv`

Synthesis Warning(s)

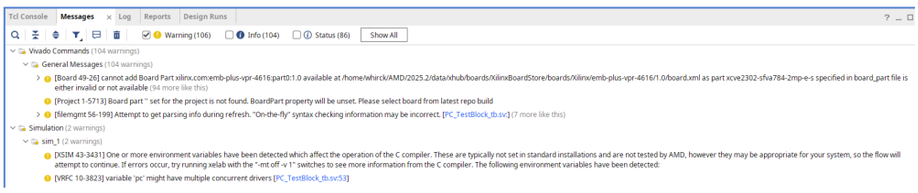

Figure 8: Synthesis Errors of `PC_TextBlock.sv`

The only error with importance here is 'clk' having multiple concurrent drivers such that one clock toggles clk and the other initially defines the clk value. I opted to proceed with this error.

Verification

All test cases passed in `PC_TestBlock_tb.sv` passed in simulation along with each subcomponent test bench.

Listing 2: Simulation file for `PC_TestBlock.sv`

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 07/19/2026 09:39:05 AM
// Design Name:
// Module Name: PC_TestBlock_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
```

```
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module PC_TestBlock_tb();

    // ~~~~ init local vars ~~~~
    logic reset, write, clk;
    logic[2:0] sel;
    logic[5:0][31:0] d;         // d[0] is unused b/c its p+4 on mux
    logic[31:0] pc, pc_plus_4;

    // ~~~~ instances ~~~~
    PC_TestBlock UUT(
        .PC_RST(reset),
        .PC_WE(write),
        .CLK(clk),
        .PC_SEL(sel),
        .JALR(d[1]),
        .BRANCH(d[2]),
        .JAL(d[3]),
        .MTVEC(d[4]),
        .MEPC(d[5]),
        .PC_COUNT(pc),
        .PC_PLUS_FOUR(pc_plus_4)
    );

     // ~~ action : CLK (10ns period) ~~
    always begin
        #5;
        clk <= !clk;
    end

    // ~~~~ action : data ~~~~
    initial begin

        // ~~ init ~~
        clk = 1; reset = 0; write = 0; sel = 0;
        d[1] = 32'h1234_5678;
        d[2] = 32'hBAAC_4444;
        d[3] = 32'h0000_01A1;
        d[4] = 32'hBEEF_BEEF;
        d[5] = 32'hDEAD_DEAD;
        pc = 0; pc_plus_4 = 0;
        #10;

        // ~~ TC1 : sel all w/ write disabled ~~
        for (int i = 0; i < 16; i++) begin
            sel = i;
            #10;
            assert(pc === 0) else $error("TC1: sel failed (pc)");
            assert(pc_plus_4 === 4) else $error("TC1: sel failed (pc+4)");
        end



        // ~~ TC2 : sel all w/ write enabled ~~
        reset = 1; sel = 0; #10; reset = 0; #10;
        write = 1; #10;

        assert(pc === 4) else $error("TC2: sel(0) failed on pc");       // Not zero b/c d[0] is 0+4 already
        assert(pc_plus_4 === 8) else $error("TC2: sel(0) failed on pc+4");

        for(int i=1; i < 6; i++) begin
            sel = i; #10;
            assert(pc === d[i]) else $error("TC2: failed on pc");
            assert(pc_plus_4 === (d[i]+4) ) else $error("TC2: failed on pc+4");
        end


        // ~~ TC3 : inc pc ~~
        reset = 1; sel = 0; #10; reset = 0;
        for(int i=1; i < 256; i++) begin
            #10;
            assert(pc === (4*i)) else $error("TC3: failed on pc");
            assert(pc_plus_4 === (4+(4*i)) ) else $error("TC3: failed on pc+4");
        end


        // ~ test conclusion ~
        reset = 1;
    end
endmodule
```

Listing 3: Simulation file for `Mux_6N.sv`

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 07/17/2026 04:26:27 PM
// Design Name:
// Module Name: Mux_6N_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module Mux_6N_tb();

    // ~~ local variables ~~
    logic CLK;
    logic[31:0]  d0, d1, d2, d3, d4, d5;
    logic[3:0]                       sel;
    logic[31:0]                      out;


    // ~~ high-level module instances (and hooks) ~~
    Mux_6N #(32) UUT (
        .D0(d0),
        .D1(d1),
        .D2(d2),
        .D3(d3),
        .D4(d4),
        .D5(d5),
        .S(sel),
        .Y(out)
    );

    // ~~ action : CLK (10ns period) ~~
    always begin
        #5;
        CLK <= !CLK;
    end

    // ~~ action : Data ~~
    initial begin

        // ~ init ~
        CLK = 0;
        d0 = 32'h0000_0000;
        d1 = 32'h1234_0000;
        d2 = 32'h0000_5678;
        d3 = 32'h1111_1111;
        d4 = 32'hFFFF_FFFF;
        d5 = 32'hDEAD_BEEF;

        // ~ selector test ~
        sel = 0; #10
        assert(out === d0) else $error("sel = 0 failed");

        sel = 1; #10
        assert(out === d1) else $error("sel = 1 failed");

        sel = 2; #10
        assert(out === d2) else $error("sel = 2 failed");

        sel = 3; #10
        assert(out === d3) else $error("sel = 3 failed");

        sel = 4; #10
        assert(out === d4) else $error("sel = 4 failed");

        sel = 5; #10
        assert(out === d5) else $error("sel = 5 failed");

        sel = 0; #10
        assert(out === d0) else $error("sel = 0 failed");

        sel = 2; #10
        assert(out === d2) else $error("sel = 2 failed");

        sel = 0; #10
        assert(out === d0) else $error("sel = 0 failed");

```

```
        sel = 4; #10
        assert(out === d4) else $error("sel = 4 failed");

        // ~ edge case : input changes ~
        sel = 3; #10
        assert(out === d3) else $error("sel = 3 failed");

        d3 = 32'hF000_0000; #10
        assert(out ===  32'hF000_0000) else $error("h7 failed");

        d3 = 32'h0F00_0000; #10
        assert(out ===  32'h0F00_0000) else $error("h6 failed");

        d3 = 32'h00F0_0000; #10
        assert(out ===  32'h00F0_0000) else $error("h5 failed");

        d3 = 32'h000F_0000; #10
        assert(out ===  32'h000F_0000) else $error("h4 failed");

        d3 = 32'h0000_F000; #10
        assert(out ===  32'h0000_F000) else $error("h3 failed");

        d3 = 32'h0000_0F00; #10
        assert(out ===  32'h0000_0F00) else $error("h2 failed");

        d3 = 32'h0000_00F0; #10
        assert(out ===  32'h0000_00F0) else $error("h1 failed");

        d3 = 32'h0000_000F; #10
        assert(out ===  32'h0000_000F) else $error("h0 failed");

        // ~ default case ~
        sel = 6;
        while(sel < 15) begin
            #10;
            assert(out === (32'hFFFF_FFFF - 13)) else $error("default %d failed", sel);
            sel = sel + 1;
        end

    end


endmodule
```

Listing 4: Simulation file for `PC_Reg.sv`

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 07/17/2026 06:36:03 PM
// Design Name:
// Module Name: PC_Reg_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module PC_Reg_tb();

    // ~~ local variables ~~
    logic clk, reset, write;
    logic[31:0] din, dout;
    logic[31:0] answer;


    // ~~ high-level module instances (and hooks) ~~
    PC_Reg UUT (
        .CLK(clk),
        .PC_RST(reset),
        .PC_WE(write),
        .PC_DIN(din),
        .PC_COUNT(dout)
    );

    // ~~ action : CLK (10ns period) ~~
    always begin
        #5;
        clk <= !clk;
    end

```

```
    // ~~ action : Data ~~
    initial begin

        // ~ init ~
        clk = 1; reset = 0; write = 0;
        reset = 1; #10;                     // zero out register

        // ~ test cases ~
        answer = 32'hDEAD_BEEF;
        din = answer;

        // ~ TC1 : write ~
        reset = 0; #10;
        assert(dout === 0) else $error("TC1: init failed");
        write = 1; #10;
        assert(dout === answer) else $error("TC1: write failed");

        // ~ TC2: reset after write ~
        reset = 1; #10;
        assert(dout === 0) else $error("TC2: reset failed");

        // ~ TC3: reset priority over write ~
        write = 1; reset = 1; #10;
        assert(dout === 0) else $error("TC3: reset failed");


        // ~ TC4 : write within range ~
        write = 1; reset = 0; #10;
        answer = 32'h0246_8ACE;
        for (int i = 0; i < 8; i++) begin
            din = answer; #10;
            assert(dout === answer) else $error("TC4: write failed");
            answer = answer << 4;
        end

        // ~ TC5 : Rollover ~
        din = 32'hFFFF_FFFF; #10;
        assert(dout === din) else $error("TC5: write failed");
        din = din + 1; #10;
        assert(dout === 0) else $error("TC5: rollover failed");




    end


endmodule
```

SystemVerilog Code

Listing 5: PC_TestBlock.sv

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 07/17/2026 07:34:25 PM
// Design Name:
// Module Name: PC_TestBlock
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module PC_TestBlock(
        input logic PC_RST, PC_WE, CLK,
        input logic[2:0] PC_SEL,
        input logic[31:0] JALR, BRANCH, JAL, MTVEC, MEPC,
        output logic[31:0] PC_COUNT, PC_PLUS_FOUR
    );

    // ~~~~ inits and wires ~~~~
    logic[31:0] MUX_0_WIRE;

    // ~~~~ instances ~~~~
    Mux_6N #(32) Mux(
        .D0(PC_PLUS_FOUR),
        .D1(JALR),
```

```
        .D2(BRANCH),
        .D3(JAL),
        .D4(MTVEC),
        .D5(MEPC),
        .S(PC_SEL),
        .Y(MUX_O_WIRE)
    );


    PC_Reg PC(
        .CLK(CLK),
        .PC_RST(PC_RST),
        .PC_WE(PC_WE),
        .PC_DIN(MUX_O_WIRE),
        .PC_COUNT(PC_COUNT)
     );

    Plus4_1N #(32) P4(
        .IN(PC_COUNT),
        .OUT(PC_PLUS_FOUR)
    );


endmodule
```

0.2 Software

0.2.1 SW1 - Intro to Assembly Lang

Note : Assume caller/callee does not need to preserve their respective registers in order to use them. KISS method applied for assignment 1.

Part 1

"Read three 16-bit unsigned values consecutively from SWITCHES (address 0x11000000), add them together, and output the result to LEDS (address 0x11000020). Assume the input values are 16-bit unsigned values. (Assume it is possible for the switches to change between each time they are read)"

Assuming the LEDS input value is limited to 16-bit unsigned as well.

Flowchart

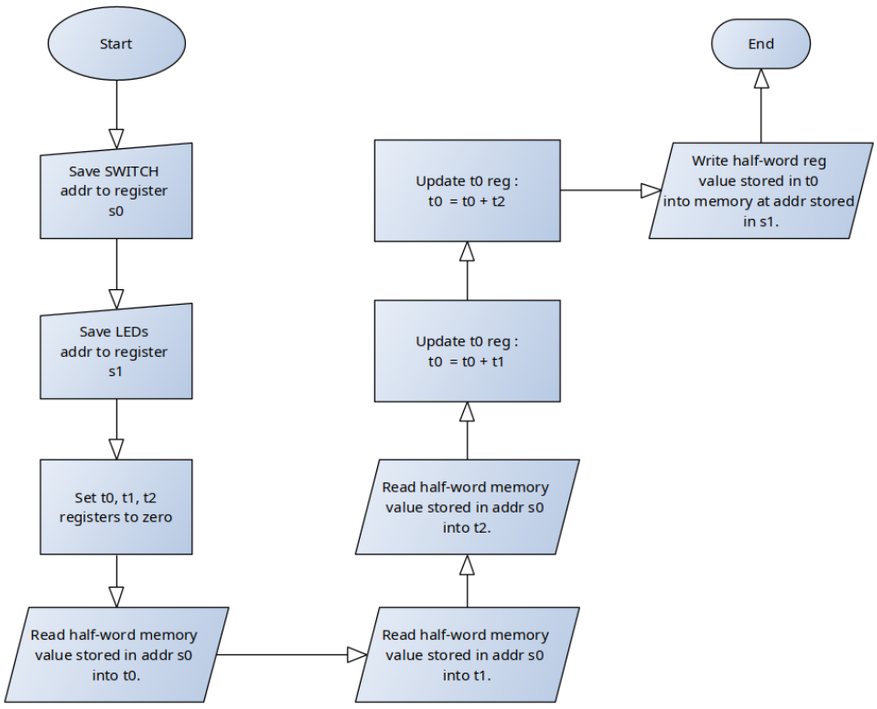

Figure 9: Flow Chart for Part 1

Verification Simulations

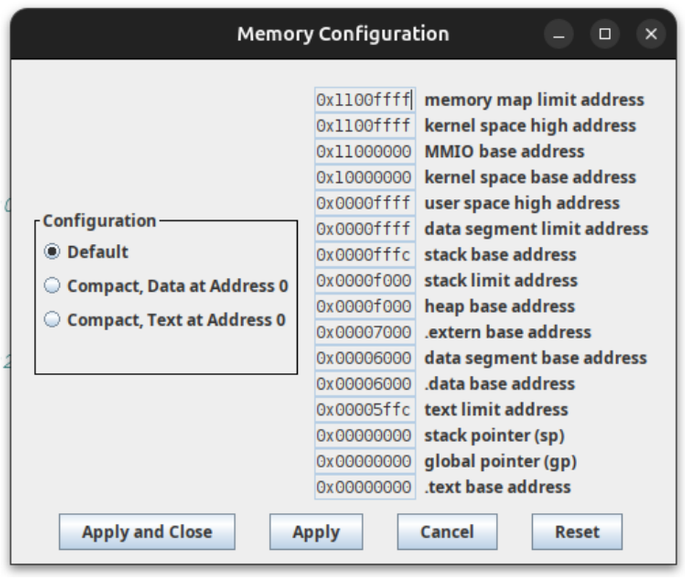

Figure 10: RARS Memory Configuration

```
20
21  #.equ SWITCHES_H,    0x11000 # upper 20 bits
22  #.equ SWITCHES_L,    0x000   # lower 12 bits
23  .equ SWITCHES_H,     0x00006 # upper 20 bits (data seg at 0x00006000)
24  .equ SWITCHES_L,     0x000   # lower 12 bits
25
26  #.equ LEDS_H,        0x11000 # upper 20 bits
27  #.equ LEDS_L,        0x020   # lower 12 bits
28  .equ LEDS_H,         0x00006 # upper 20 bits (data seg at 0x00006020)
29  .equ LEDS_L,         0x020   # lower 12 bits
30
```

Figure 11: Address Correction to work with RARS

- **Test Case 1**

```
33  # TC1 : Word Clip
34  .equ READ1_H,        0x00001 # switch first read (0x1234)
35  .equ READ1_L,        0x234
36
37
38  .equ READ2_H,        0x12345 # switch second read (0x5678)
39  .equ READ2_L,        0x678
40
41  .equ READ3_H,        0x11111 # switch third read (0x1111)
42  .equ READ3_L,        0x111
43  #    answer= 0x79BD
44
```

Figure 12: TC1 given inputs

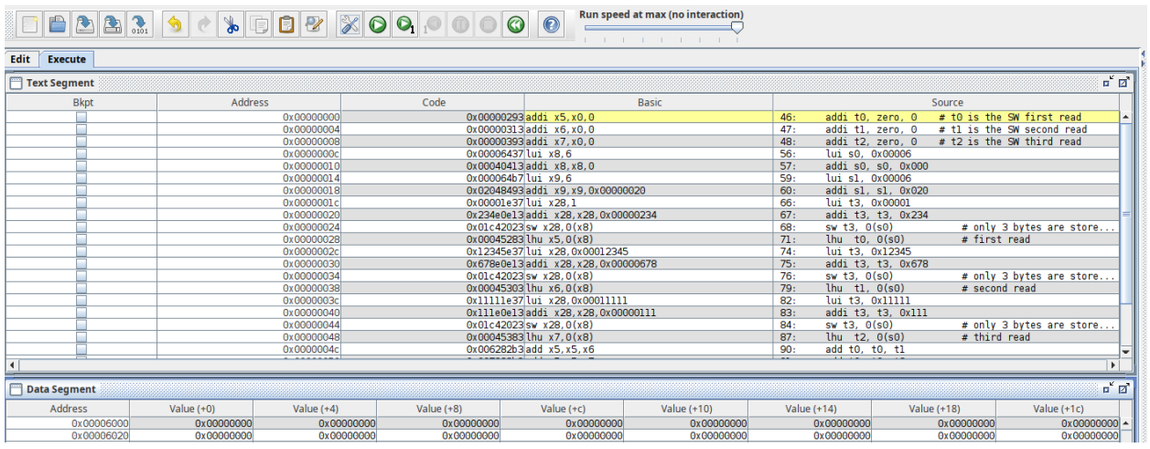

Figure 13: Data segment before executing instructions

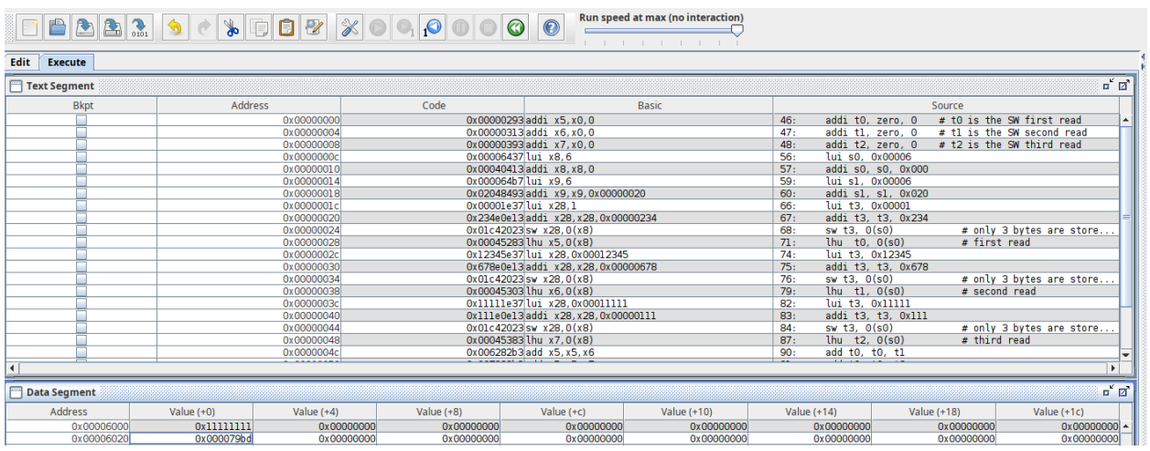

Figure 14: Data segment after executing instructions

Result at LEDS address in 14 matches the answer in 12.

- **Test Case 2**

```
46  # TC2 : Maxed
47  .equ READ1_H,        0x00005 # switch first read (0x5555)
48  .equ READ1_L,        0x555
49
50
51  .equ READ2_H,        0x00005 # switch second read (0x5555)
52  .equ READ2_L,        0x555   # sign extended
53
54  .equ READ3_H,        0x00005 # switch third read (0x5555)
55  .equ READ3_L,        0x555
56  #   answer= 0xFFFF
```

Figure 15: TC2 given input

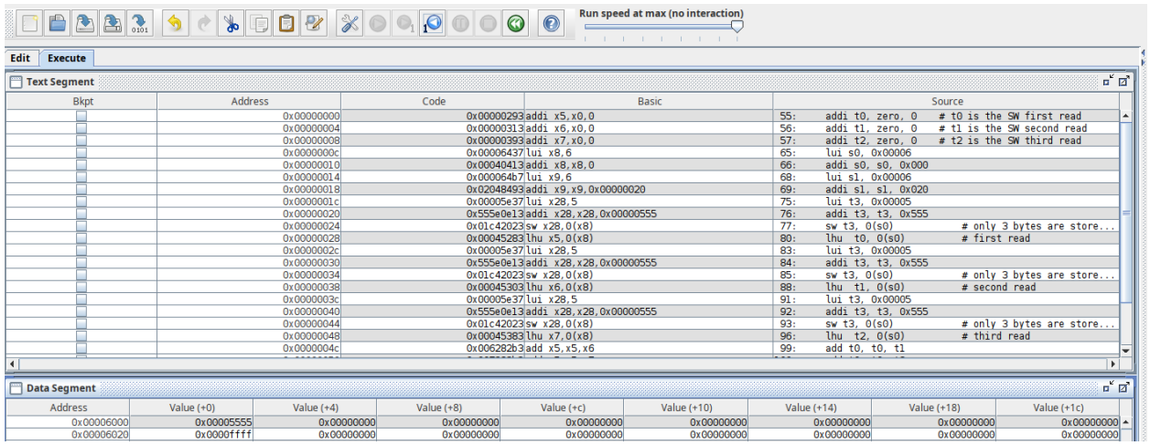

Figure 16: Data segment after executing instructions

Result at LEDS address in 16 matches the answer in 15.

- **Test Case 3**

```
59  # TC3 : Overflow
60  .equ READ1_H,        0x00005 # switch first read (0x5555)
61  .equ READ1_L,        0x555
62
63  .equ READ2_H,        0x00005 # switch second read (0x5555)
64  .equ READ2_L,        0x555   # sign extended
65
66
67  .equ READ3_H,        0x00005 # switch third read (0x5557)
68  .equ READ3_L,        0x557
69  #   answer= 0x0001
```

Figure 17: TC3 given input

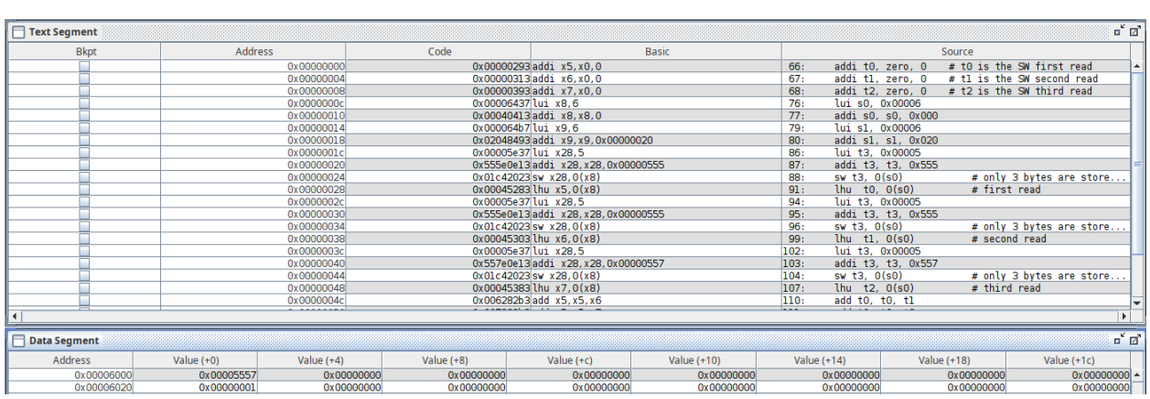

Figure 18: Data segment after executing instructions

Result at LEDS address in 18 matches the answer in 17.

- **Test Case 4**

```
71  # TC4 : zero
72  .equ READ1_H,        0x00000 # switch first read (0)
73  .equ READ1_L,        0x0
74
75
76  .equ READ2_H,        0x00000 # switch second read (0)
77  .equ READ2_L,        0x0 # sign extended
78
79  .equ READ3_H,        0x00000 # switch third read (0)
80  .equ READ3_L,        0x0
81  #    answer= 0x0000
82  "
```

Figure 19: TC4 given input

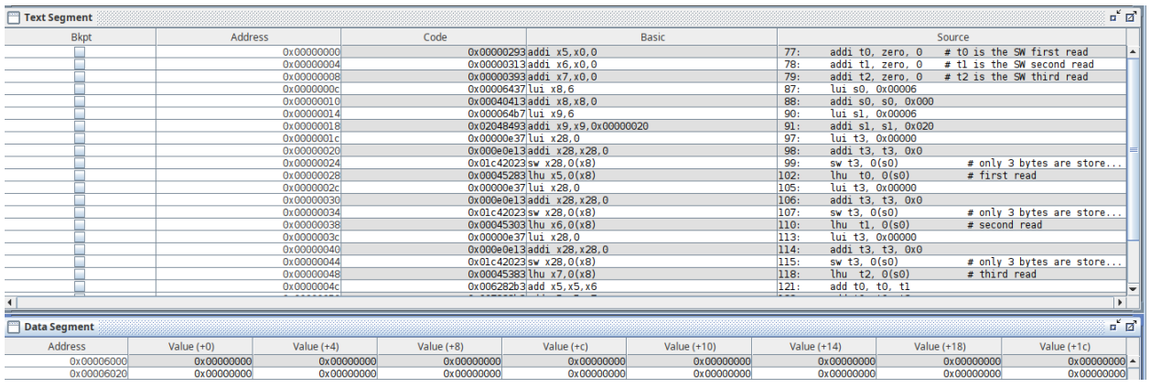

Figure 20: Data segment after executing instructions

Result at LEDS address in 20 matches the answer in 19.

- **Test Case 5**

```
84  # TC5 : MAXXXED
85  .equ READ1_H,        0x0000F # switch first read (0xFFFF)
86  .equ READ1_L,        -1
87
88  .equ READ2_H,        0xFFFFF # switch second read (0xFFFF)
89  .equ READ2_L,        -1
90
91
92  .equ READ3_H,        0x00000 # switch third read (0)
93  .equ READ3_L,        0
94  #    answer= 0xFFFE
```

Figure 21: TC5 given input

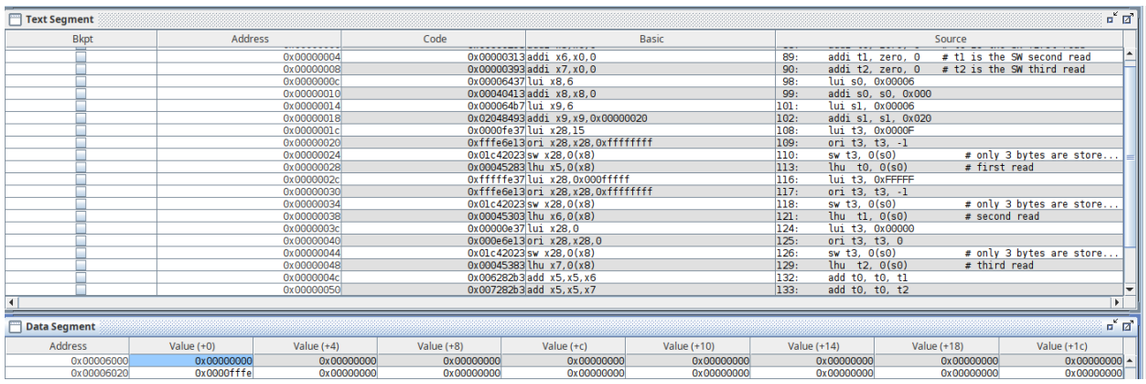

Figure 22: Data segment after executing instructions

Result at LEDS address in 22 matches the answer in 21.

- **Test Case 6**

```
96
97  # TC6 : All highs into full
98  .equ READ1_H,        0x00008 # switch first read
99  .equ READ1_L,        0x000
100
101 .equ READ2_H,        0x00008 # switch second read
102 .equ READ2_L,        0x000
103
104
105 .equ READ3_H,        0x00008 # switch third read
106 .equ READ3_L,        0x000
107 #    answer= 0x8000 (0x18000 as full word though)
```

Figure 23: TC6 given input

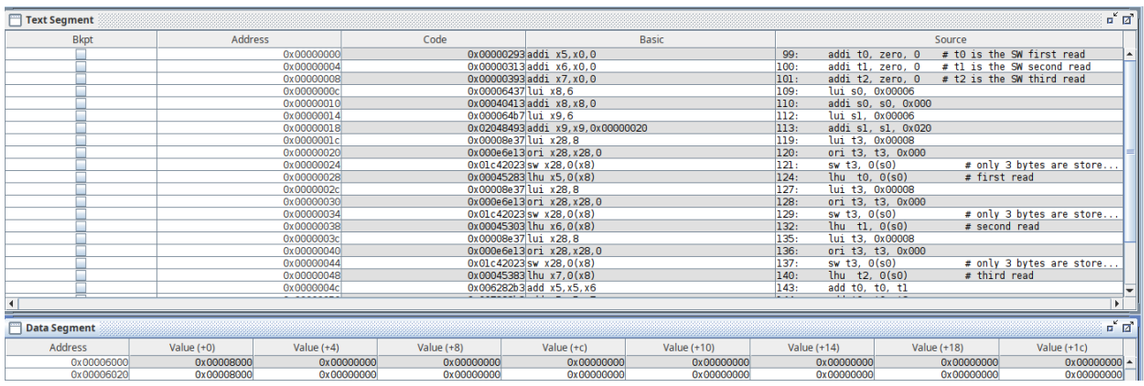

Figure 24: Data segment after executing instructions

Result at LEDS address in 24 matches the answer in 23.

Assembly Source Code

Listing 6: Question 1 of SW1

```
1  # Author : WhisperingRock
2  #
3  # Purpose : Read from the SWITCHES addr in memory three consecutive
4  #      times, summing the unsigned half-word reads and writing
5  #      to the LEDS addr.
6  #
7  # Notes :
8  #      - Assume caller/calle has no need to preserve reg values.
9  #        KISS method for this assignment
10 #
11 #      - Split 32-bit addresses according to instr imm handling.
12
13
14
15
16 .global main
17
18 #.equ SWITCHES_H, 0x11000 # upper 20 bits
19 #.equ SWITCHES_L, 0x000 # lower 12 bits
20 .equ SWITCHES_H,  0x00006 # upper 20 bits (data seg at 0x00006000)
21 .equ SWITCHES_L,  0x000 # lower 12 bits
22
23 #.equ LEDS_H,    0x11000 # upper 20 bits
24 #.equ LEDS_L,    0x020 # lower 12 bits
25 .equ LEDS_H,     0x00006 # upper 20 bits (data seg at 0x00006020)
26 .equ LEDS_L,     0x020 # lower 12 bits
27
28 # ~~~~~~~~ Test Cases ~~~~~~~~
29 # TC1 : Word Clip
30 #.equ READ1_H,     0x00001 # switch first read (0x1234)
31 #.equ READ1_L,     0x234
32
33 #.equ READ2_H,     0x12345 # switch second read (0x5678)
34 #.equ READ2_L,     0x678
35
36 #.equ READ3_H,     0x11111 # switch third read (0x1111)
37 #.equ READ3_L,     0x111
38 # answer= 0x79BD
39
40 # TC2 : Maxed
41 #.equ READ1_H,     0x00005 # switch first read (0x5555)
```

```
#.equ READ1_L,    0x555

#.equ READ2_H,    0x00005 # switch second read (0x5555)
#.equ READ2_L,    0x555 # sign extended

#.equ READ3_H,    0x00005 # switch third read (0x5555)
#.equ READ3_L,    0x555
# answer= 0xFFFF

# TC3 : Overflow
#.equ READ1_H,    0x00005 # switch first read (0x5555)
#.equ READ1_L,    0x555

#.equ READ2_H,    0x00005 # switch second read (0x5555)
#.equ READ2_L,    0x555 # sign extended

#.equ READ3_H,    0x00005 # switch third read (0x5557)
#.equ READ3_L,    0x557
# answer= 0x0001

# TC4 : zero
#.equ READ1_H,    0x00000 # switch first read (0)
#.equ READ1_L,    0x0

#.equ READ2_H,    0x00000 # switch second read (0)
#.equ READ2_L,    0x0 # sign extended

#.equ READ3_H,    0x00000 # switch third read (0)
#.equ READ3_L,    0x0
# answer= 0x0000

# TC5 : MAXXXED
#.equ READ1_H,    0x0000F # switch first read (0xFFFF)
#.equ READ1_L,    -1

#.equ READ2_H,    0xFFFFF # switch second read (0xFFFF)
#.equ READ2_L,    -1

#.equ READ3_H,    0x00000 # switch third read (0)
#.equ READ3_L,    0
# answer= 0xFFFE

# TC6 : All highs into full
.equ READ1_H,   0x00008 # switch first read
.equ READ1_L,     0x000

.equ READ2_H,     0x00008 # switch second read
.equ READ2_L,     0x000

.equ READ3_H,   0x00008 # switch third read
.equ READ3_L,   0x000
# answer= 0x8000 (0x18000 as full word though)
# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

main:

  # ~~~~ register init ~~~~
  addi t0, zero, 0  # t0 is the SW first read
  addi t1, zero, 0  # t1 is the SW second read
  addi t2, zero, 0  # t2 is the SW third read
          # t3 is a dummy variable for testing

          # s0 contains the SWITCH addr
          # s1 contains the LEDS addr


  # ~~~~ load addr into registers ~~~~
  lui s0, SWITCHES_H
  addi s0, s0, SWITCHES_L

  lui s1, LEDS_H
  addi s1, s1, LEDS_L


  # ~~~~ read SWITCHES and sum ~~~~

  # ~~ behind the scenes : switch mem changes value ~~
  lui t3, READ1_H
  ori t3, t3, READ1_L
  sw t3, 0(s0)      # only 3 bytes are stored (0x234)
  # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

  lhu  t0, 0(s0)      # first read

  # ~~ behind the scenes : switch mem changes value ~~
  lui t3, READ2_H
  ori t3, t3, READ2_L
  sw t3, 0(s0)      # only 3 bytes are stored (0x678)
  # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

  lhu  t1, 0(s0)      # second read
```

```
133
134 # ~~~~ behind the scenes : switch mem changes value ~~~~
135 lui t3, READ3_H
136 ori t3, t3, READ3_L
137 sw t3, 0(s0)       # only 3 bytes are stored (0x111)
138 # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
139
140 lhu  t2, 0(s0)       # third read
141
142 # ~~~~ sum values ~~~~
143 add t0, t0, t1
144 add t0, t0, t2
145
146
147 # ~~~~ Write the unsigned half word to LEDS mem addr
148 sh t0, 0(s1)
149
150
151 # ~~~~ return with exit status ~~~~
152 li a7, 10    # load system call number for exit()
153 li a0, 0     # Load the exit status code ( EXIT_SUCCESS )
154 ecall      # Make the system call
```

Part 2

"Read an input from SWITCHES (address 0x11000000). Assume the input is a 16-bit signed value in 2's complement (RC). Change the sign of the input and output the result to LEDS (address 0x11000020). The result should also be a 16-bit signed value in 2's complement (RC)."

Flowchart

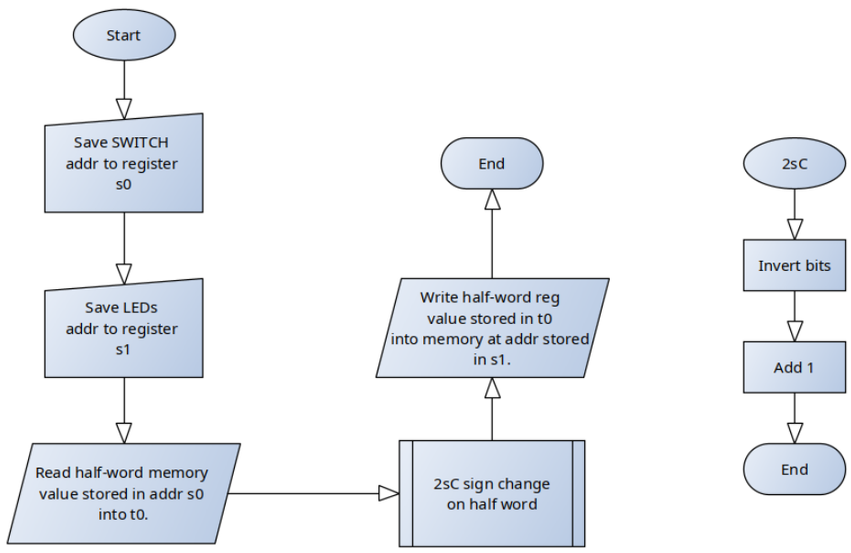

Figure 25: Flow Chart for Part 2

Verification Simulations

Test cases passed with definitions defined in code below.

```
16 # TC1 : zero
17 #A: .half 0
18 #    answer= 0x0000
```

Data Segment

Address	Value (+0)
0x00006000	0x00000000
0x00006020	0x00000000

Figure 26: IO and Data Segment of TC1

```
21 # TC2 : positive
22 #A: .half 0x5
23 #    answer= 0xFFFB (-5)
```

Data Segment

Address	Value (+0)
0x00006000	0x00000005
0x00006020	0x0000fffb

Figure 27: IO and Data Segment of TC2

```
# TC3 : negative
#A: .half 0xFFFB         # -5
#   answer= 0x0005       #  5
```

Data Segment

Address	Value (+0)
0x00006000	0x0000fffb
0x00006020	0x00000005

Figure 28: IO and Data Segment of TC3

```
# TC4 : positive
#A: .half 0x5678         # 22136
#   answer= 0xA988       # -22136
```

Data Segment

Address	Value (+0)
0x00006000	0x00005678
0x00006020	0x0000a988

Figure 29: IO and Data Segment of TC4

```
# TC5 : reverse
#A: .half 0xA988
#   answer= 0x5678
```

Data Segment

Address	Value (+0)
0x00006000	0x0000a988
0x00006020	0x00005678

Figure 30: IO and Data Segment of TC5

```
# TC6 : negative
#A : .half 0xFFFF #(-1)
#   answer= 1
```

Data Segment

Address	Value (+0)
0x00006000	0x0000ffff
0x00006020	0x00000001

Figure 31: IO and Data Segment of TC6

```
# TC7 : negative 2
#A : .half 0xFFFE #(-2)
#   answer= 2
```

Data Segment

Address	Value (+0)
0x00006000	0x0000fffe
0x00006020	0x00000002

Figure 32: IO and Data Segment of TC7

```
# TC7 : negative 32 bit
#A : .word 0x89ABCDEF       #(0xCDEF = -12817)
#   answer= 0x3211          # 12817
```

Data Segment

Address	Value (+0)
0x00006000	0x89abcdef
0x00006020	0x00003211

Figure 33: IO and Data Segment of TC8

Assembly Code

Listing 7: Question 2 of SW1

```
# Author : WhisperingRock
#
# Purpose : Read from the SWITCHES addr in memory a unsigned half-word
#     changing the sign, then writing to the LEDS addr.
#
# Notes :
#     - Assume caller/calle has no need to preserve reg values.
#       KISS method for this assignment
#
#     - Split 32-bit addresses according to instr imm handling.

.data
# ~~~~~~~~ Test Cases ~~~~~~~~
# TC1 : zero
#A: .half 0
# answer= 0x0000

# TC2 : positive
#A: .half 0x5
# answer= 0xFFFB (-5)

# TC3 : negative
#A: .half 0xFFFB     # -5
# answer= 0x0005     #  5

# TC4 : positive
#A: .half 0x5678     # 22136
```

```
# answer= 0xA988    # -22136

# TC5 : reverse
#A: .half 0xA988
# answer= 0x5678

# TC6 : negative
#A : .half 0xFFFF #(-1)
# answer= 1

# TC7 : negative 2
#A : .half 0xFFFE #(-2)
# answer= 2

# TC7 : negative 32 bit
#A : .word 0x89ABCDEF   #(0xCDEF = -12817)
# answer= 0x3211      # 12817

# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~


.global _start
.text
#.equ SWITCHES_H, 0x11000 # upper 20 bits
#.equ SWITCHES_L, 0x000 # lower 12 bits
.equ SWITCHES_H,  0x00006 # upper 20 bits (data seg at 0x00006000)
.equ SWITCHES_L,  0x000 # lower 12 bits

#.equ LEDS_H,   0x11000 # upper 20 bits
#.equ LEDS_L,   0x020 # lower 12 bits
.equ LEDS_H,    0x00006 # upper 20 bits (data seg at 0x00006020)
.equ LEDS_L,    0x020 # lower 12 bits



_start:

  # ~~~~ register init ~~~~
  addi t0, zero, 0  # t0 is the SW first read
            # t3 is a dummy variable for testing

            # s0 contains the SWITCH addr
            # s1 contains the LEDS addr

  # ~~~~ load addr into registers ~~~~
  lui s0, SWITCHES_H
  addi s0, s0, SWITCHES_L

  lui s1, LEDS_H
  addi s1, s1, LEDS_L


  # ~~~~ read SWITCHES and sum ~~~~

  # ~~ behind the scenes : switch value ~~
  lw t3, A
  sw t3, 0(s0)
  # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

  lhu  t0, 0(s0)      # first read


  # ~~~~ invert the sign (2sC) ~~~~
  not t0, t0
  addi t0, t0, 1

  # ~~~~ Write the unsigned half word to LEDS mem addr
  sh t0, 0(s1)


  # ~~~~ return with exit status ~~~~
  li a7, 10    # load system call number for exit()
  li a0, 0     # Load the exit status code ( EXIT_SUCCESS )
  ecall      # Make the system call
```

0.2.2 SW2 - Conditional Statements

Assignment:
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

1. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input is greater than or equal to 32,768, the value is divided by 4. You can ignore any remainder. If the value is less than 32,768, the value is multiplied by 2. The result should be written to the 7 SEGMENT (address 0x11000040).
2. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input value is a multiple of 4, all of the bits should be inverted, otherwise, if the input value is odd, add 4095 and divide the result by 2, otherwise subtract 1 from the value. The result should be written to the 7 SEGMENT (address 0x11000040).

Assume the BASYS3 SWITCHES are expanded to 32 individual switches and 7 SEGMENT is expanded to 8 digits.

Figure 34: Questions to address

Part 1

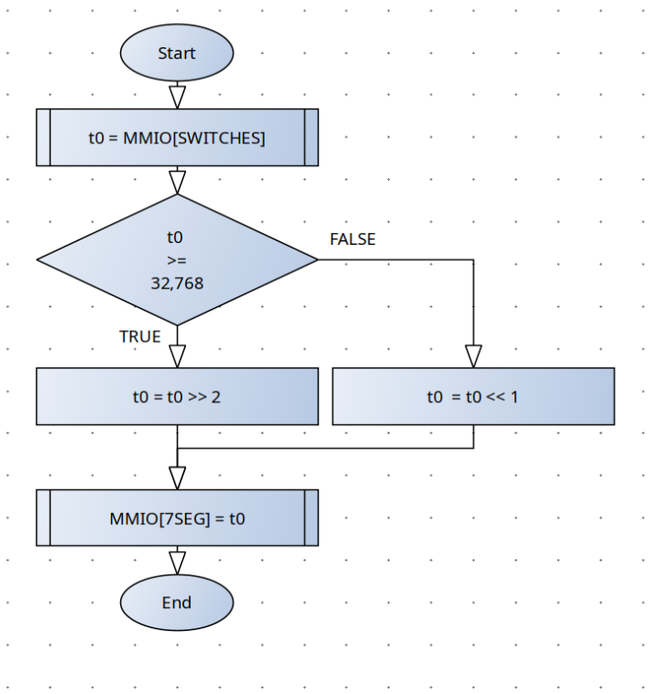

Figure 35: Flow chart for part 1

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Lowest unsigned val	0	0	Passed
2	Below boundary	$0xFFF$	$0x1FFE$	Passed
3	1 before boundary	$0x7FFF$	$0xFFFE$	Passed
4	On boundary	$0x8000$	$0x2000$	Passed
5	1 above boundary	$0x8001$	$0x2000$	Passed
6	Above boundary	$0x12345678$	$0x048D159E$	Passed
7	32-bit unsigned ceiling	$0xFFFFFFFF$	$0x3FFFFFFF$	Passed

Table 2: Test cases for part 1

Listing 8: sw2_p1.asm

```
# Author : WhisperingRock
#
# Purpose : Conditionals, branching, jumping
# Input :
#     SWITCHES MMIO address to 32-bit unsigned value
#     7SEG MMIO address to 8-digits, each of 4-bits
#     constant CONST
#
# Output :
#     Write to 7SEG addr


.data
# ~~~~ Test Cases ~~~~

# ~~ TC1 : Zero ~~
#TC:    .word 0
# answer = 0 (verified)

# ~~ TC2 : Under ~~
#TC:    .word 0xFFF
# answer = 0x1FFE (verified)

# ~~ TC3 : One before ~~
#TC:    .word 0x07FFF
# answer = 0xFFFE (verified)

# ~~ TC4 : On boundary ~~
#TC:    .word 0x08000
# answer = 0x2000 (verified)

# ~~ TC5 : Above boundary ~~
#TC:    .word 0x08001
# answer = 0x2000 (verified)

# ~~ TC6 : Above boundary 2 ~~
#TC:    .word 0x12345678
# answer = 0x048D159E (verified)

# ~~ TC7 : 32-bit unsigned ceiling ~~
TC:   .word 0xFFFFFFFF
# answer = 0x3FFFFFFF (verified)

.global _start

.text
.equ MMIO_SWITCHES, 0x11000000
.equ MMIO_SEVSEG, 0x11000040
.equ CONST, 32768

_start:
  # ~~~~ init reg ~~~~
  li t1, MMIO_SWITCHES    # t1 is the addr for MMIO SWITCHES
  li t2, CONST        # t2 is the constant 32768
  li t3, MMIO_SEVSEG      # t3 is the addr for MMIO 7SEG
                # t4 is a test case variable

  # ~~~~ TC injection ~~~~
  lw t4, TC
  sw t4, 0(t1)

  # ~~~~ read SWITCHES ~~~~
  lw t0, 0(t1)        # load SW value into t0

  # ~~~~ unsigned branch ~~~~~
  bgeu t0, t2, Div4

  # ~~ else (false) case ~~
  slli t0, t0, 1        # multiply by 2
  j End

  # ~~ if (true) case ~~
Div4:
  srli t0, t0, 2        # logical shift for unsigned

  # ~~~~ write to 7SEG ~~~~
End:
  sw t0, 0(t3)

  # ~~~~ syscall exit status ~~~~
  li a7, 10     # load syscall num for exit()
  li a0, 0      # EXIT_SUCCESS code
  ecall       # execute sys call


```

Figure 36: Assembly code for SW2 part 1

Part 2

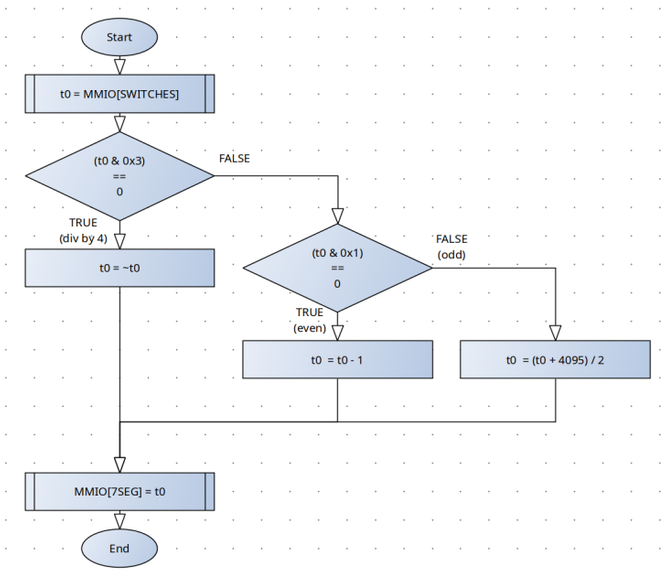

Figure 37: Flow chart for part 2

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Zero is div by 4	0	$0xFFFFFFFF$	Passed
2	Two is even	$0x2$	$0x1$	Passed
3	One is odd	$0x1$	$0x0800$	Passed
4	32-bit unsinged ceiling	$0xFFFFFFFF$	$0x07FF$	Passed
5	Div by 4 near ceiling	$0xFFFFFFFC$	$0x3$	Passed
6	Odd near ceiling	$0xFFFFFFFD$	$0x07FE$	Passed
7	Even near ceiling	$0xFFFFFFFE$	$0xFFFFFFFD$	Passed

Table 3: Test cases for part 1

Listing 9: sw2_p2.asm

```
# Author : WhisperingRock
#
# Purpose : Conditionals, branching, jumping
# Input :
#     SWITCHES MMIO address to 32-bit unsigned value
#     7SEG MMIO address to 8-digits, each of 4-bits
#     constant CONST
#
# Output :
#     Write to 7SEG addr


.data
# ~~~~ Test Cases ~~~~

# ~~ TC1 : Zero (Div4) ~~
#TC:    .word 0
# answer = 0xFFFF_FFFF  (verified)

# ~~ TC2 : Two (Div2 = even) ~~
#TC:    .word 2
# answer = 0x1      (verified)

# ~~ TC3 : 1 (odd) ~~
#TC:    .word 0x1
# answer = 0x0800   (verified)

# ~~ TC4 : 32-bit unsigned ceiling ~~
#TC:    .word 0xFFFFFFFF
# answer = 0x0000_07FF | 2047 (verified)

# ~~ TC5 : unsigned ceiling Div4 ~~
#TC:    .word 0xFFFFFFFC
# answer = 0x03         (verified)

# ~~ TC6 : unsigned ceiling odd ~~
#TC:    .word 0xFFFFFFFD
# answer = 0x07FE         (verified)

# ~~ TC6 : unsigned ceiling even ~~
TC:   .word 0xFFFFFFFE
# answer = 0xFFFFFFFD       (verified)

.global _start

.text
.equ MMIO_SWITCHES, 0x11000000
.equ MMIO_SEVSEG, 0x11000040


_start:
  # ~~~~ init reg ~~~~
  li s1, MMIO_SWITCHES    # s1 is the addr for MMIO SWITCHES
  li s2, MMIO_SEVSEG      # s2 is the addr for MMIO 7SEG
                # t0 is the working variable for SWITCHES value
                # t1 is a logic case variable
  li t2, 4095         # t2 is reserved for const

  # ~~~~ TC injection ~~~~
  lw t0, TC
  sw t0, 0(s1)

  # ~~~~ read SWITCHES ~~~~
  lw t0, 0(s1)        # load SW value into t0

  # ~~~~ branch : div by 4 ~~~~~
  andi t1, t0, 0x3
  beqz t1, Div4

  # ~~~~ branch : div by 2 ~~~~~
  andi t1, t0, 0x1
  beqz t1, Div2

  # ~~ else case ~~
  add t0, t0, t2        # add const 4095
  srli t0, t0, 1        # then scale
  j End

Div4:
  not t0, t0          # invert bits
  j End

Div2:
  addi t0, t0, -1

End:  # ~~~~ write to 7SEG ~~~~
  sw t0, 0(s2)



  # ~~~~ syscall exit status ~~~~
  li a7, 10     # load syscall num for exit()
  li a0, 0      # EXIT_SUCCESS code
  ecall       # execute sys call


```

0.2.3 SW3 - Loops

Assignment:
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

1. Read a 32-bit value from SWITCHES (address 0x11000000). Assume the 32-bit value is the combination of two 16-bit unsigned values. Split the 32-bit input into two 16-bit values (the upper and lower 16-bits). Assume both of the 16-bit values are unsigned. Multiply the two 16-bit values together and output the result as a 32-bit unsigned value to 7 SEGMENT (address 0x11000040).

2. Read a 32-bit value from SWITCHES (address 0x11000000), delay for 0.5 seconds, and then output the value to 7 SEGMENT (address 0x11000040). The OTTER MCU operates at 25 MHz so each instruction takes 40 ns to run. (*Hint: Use loops.*) **There is no method to verify the run time in RARS and RARS does not execute at the same speed as the OTTER.** You will need to verify your timing by showing your calculation for the number of instructions being performed between the load (reading from the switches) and store (writing to the 7 segment display) instructions.

Assume the BASYS3 SWITCHES are expanded to 32 individual switches and 7 SEGMENT is expanded to 8 digits.

Figure 39: Questions to address

Part 1

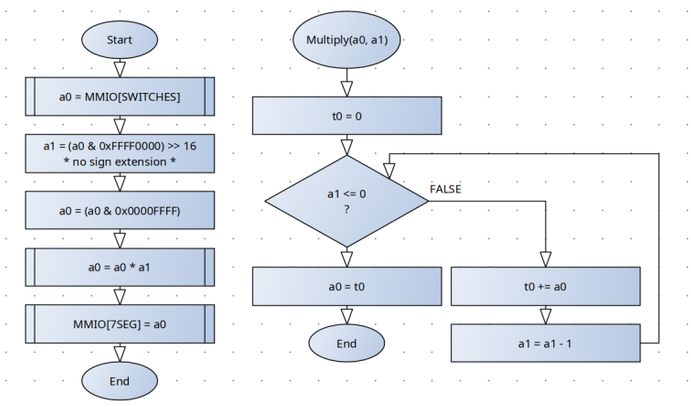

Figure 40: Flow chart for part 1

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Zero	$0x0000\_0000$	$0x0000\_0000$	Passed
2	Zero + One	$0x0000\_0001$	$0x0000\_0000$	Passed
3	One + Zero	$0x0001\_0000$	$0x0000\_0000$	Passed
4	One + One	$0x0001\_0001$	$0x0000\_0001$	Passed
5	num1	$0x0013\_0004$	$0x0000\_004C$	Passed
6	num1 reverse	$0x0004\_0013$	$0x0000\_004C$	Passed
7	shift 1	$0xFFFF\_FFFF$	$0xFFFE\_0001$	Passed
8	shift 2	$0x0010\_FFFF$	$0x000F\_FF0$	Passed
9	shift 3	$0x0100\_FFFF$	$0x00FF\_FF00$	Passed

Table 4: Test cases for part 1

Listing 10: sw3_p1.asm

```
# Author : WhisperingRock
#
# Purpose : Half word multiply
# Input :
#     SWITCHES MMIO address to 32-bit unsigned value
#     7SEG MMIO address to 8-digits, each of 4-bits
#
# Output :
#     Write to 7SEG addr


.data
# ~~~~ Test Cases ~~~~

# ~~ TC1 : Zero ~~
#TC:    .word 0
# answer = 0    (verified)
```

```
18
19  # ~~ TC2 : Zero + One ~~
20  #TC:    .word 0x00000001
21  # answer = 0    (verified)
22
23  # ~~ TC3 : One + Zero~~
24  #TC:    .word 0x00010000
25  # answer = 0    (verified)
26
27  # ~~ TC4 : One ~~
28  #TC:    .word 0x00010001
29  # answer = 1    (verified)
30
31  # ~~ TC5 : num1 ~~
32  #TC:    .word 0x00130004
33  # answer = 0x0000_004C  (verified)
34
35  # ~~ TC6 : num1 reverse ~~
36  #TC:    .word 0x00040013
37  # answer = 0x0000_004C  (verified)
38
39  # ~~ TC7 : shift1 ~~
40  #TC:    .word 0xFFFFFFFF
41  # answer = 0xFFFE_0001  (verified)
42
43  # ~~ TC8 : shift2 ~~
44  #TC:    .word 0x0010FFFF
45  # answer = 0x000F_FFF0  (verified)
46
47  # ~~ TC9 : shift3 ~~
48  TC:   .word 0x0100FFFF
49  # answer = 0x00FF_FF00  (verified)
50
51  .global _start
52
53  .text
54  #.equ MMIO_SWITCHES, 0x11000000 # moved to Compact(Text @ 0) config
55  #.equ MMIO_SEVSEG,  0x11000040  # sp moved tp 0x00003FFC
56  .equ MMIO_SWITCHES, 0x00007F00
57  .equ MMIO_SEVSEG,   0x00007F40
58  .equ UPPER_WORD,  0xFFFF0000
59  .equ LOWER_WORD,  0x0000FFFF
60
61  _start:
62    # ~~~~ init reg ~~~~
63    li s1, MMIO_SWITCHES    # s1 is the addr for MMIO SWITCHES
64    li s2, MMIO_SEVSEG      # s2 is the addr for MMIO 7SEG
65                  # a0 is a working variable for input and return
66                  # a1 is a working var for upper half of word
67                  # a2 is a working var for lower half of word
68                  # t0 is single use TC injection var
69
70    # ~~~~ TC injection ~~~~
71    lw t0, TC
72    sw t0, 0(s1)
73
74    # ~~~~ read SWITCHES ~~~~
75    lw a0, 0(s1)
76
77    # ~~~~ parse input val (and load args for func) ~~~~
78    # ~~ a1 = (a0 & 0xFFFF0000) >> 16 ~~
79    li t0, UPPER_WORD
80    and a1, a0, t0
81    srli a1, a1, 16
82    # ~~ a0 = a0 & 0x0000FFFF ~~
83    li t0, LOWER_WORD
84    and a0, a0, t0
85
86    # ~~~~ caller(_start) saves non-preserved ~~~~
87    # "Not needed b/c we dont need t0"
88    jal Multiply
89
90    # ~~~~ write return to 7SEG ~~~~
91    sw a0, 0(s2)
92
93    # ~~~~ syscall exit ~~~~
94    li a7, 10     # load syscall num for exit()
95    li a0, 0      # EXIT_SUCCESS code
96    ecall       # execute sys call
97
98
99  # ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
100 # Purpose : Multiply two unsigned 32-bit numbers
101 # Input :
102 #     a0 : 32-bit unsigned word
103 #     a1 : 32-bit unsigned word
104 # Temps :
105 #     t0 :
106 # Note  :
107 #     - consumes a1
108 # Return:
```

```
#      a0 : product of a0 * a1
Multiply:

  # ~~~~ calle(Multiply) saves preserved ~~~~
  # ~~ 1. calle(Multiply) allocates space on stack ~~
  addi sp, sp, -12

  # ~~ 2. calle(Multiply) stores reg values on stack ~~
  sw ra, 8(sp)
  sw s1, 4(sp)
  sw s2, 0(sp)

  # ~~ 3. calle(Multiply) function execution ~~
  # ~ sum(t0) init ~
  addi t0, zero, 0
  # ~ while a1 <= 0
While:
  bleu a1, zero, EndWhile
  add t0, t0, a0
  addi a1, a1, -1
  j While
EndWhile:
  addi a0, t0, 0

  # ~~ 4. calle(Multiply) restores preserved from stack ~~
  lw ra, 8(sp)
  lw s1, 4(sp)
  lw s2, 0(sp)

  # ~~ 5. calle(Multiply) deallocates space on stack ~~
  addi sp, sp, 12

  # ~~ calle(Multiply) return ~~
  jr ra
#~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

Part 2

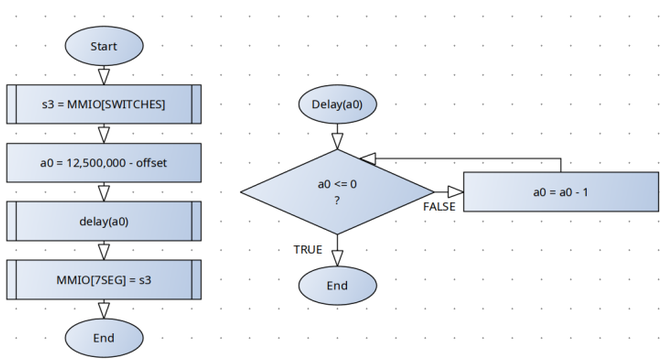

Figure 41: Flow chart for part 2

Consider the following calculations

$$\begin{aligned} 12.5 \cdot 10^6 \text{ instr} &= 0.5s \cdot \frac{\text{instr}}{40ns} \cdot \frac{10^9 ns}{1s} \\ &= 0\text{x}00\text{BE\_BC}20 \end{aligned}$$

- There are 13 instructions, after reading from SWTICHES, that are only executed once. Lets refer to these instructions as bias s.t. $b = 13$.
- There are 3 instruction in our delay loop s.t. $m = 3$.

Now lets relate our instructions to the instruction layout s.t. we identify how many times we actually need to loop.

$$\begin{aligned} 3m + b &= 12.5 \cdot 10^9 \\ m &= \frac{12,500,000 - 13}{3} \\ &= 4,166,663 \end{aligned}$$

Rather than use this number in directly, I opted to write the constant in terms of total instructions needed against how many loops we need.

$$\begin{aligned}\text{offset} &= 12.5 \cdot 10^9 - m \\ &= 8,333,337 \\ &= \text{0x007F\_2819}\end{aligned}$$

Listing 11: Assembly code for sw3_p2.asm

```
# Author : WhisperingRock
#
# Purpose : Delay
# Input :
#     SWITCHES MMIO address to 32-bit unsigned value
#     7SEG MMIO address to 8-digits, each of 4-bits
#
# Output :
#     Write to 7SEG addr


.data
# ~~~~ Test Cases ~~~~

# ~~ TC1 : simple ~~
TC:   .word 0x12345678
# answer = 0    (verified)

.global _start

.text
#.equ MMIO_SWITCHES,  0x11000000  # moved to Compact(Text @ 0) config
#.equ MMIO_SEVSEG,    0x11000040  # sp moved tp 0x00003FFC
.equ MMIO_SWITCHES,   0x00007F00
.equ MMIO_SEVSEG,     0x00007F40
.equ HALF_SEC_DELAY,  0x00BEBC20  # 12,500,000 instr for 0.5s delay
.equ OFFSET,      0x007F2819  # loop and bias compensation (see notes)

_start:
  # ~~~~ init reg ~~~~
  li s1, MMIO_SWITCHES    # s1 is the addr for MMIO SWITCHES
  li s2, MMIO_SEVSEG      # s2 is the addr for MMIO 7SEG
                # s3 is a working variable for input and return
  li a0, HALF_SEC_DELAY   # a0 is reserved for const
  li a1, OFFSET       # a1 is reserved for offset
                # t0 is single use TC injection var

  # ~~~~ TC injection ~~~~
  lw t0, TC
  sw t0, 0(s1)

  # ~~~~ read SWITCHES ~~~~
  lw s3, 0(s1)

  # ~~~~ caller(_start) saves non-preserved ~~~~
  # "Not needed b/c we dont need t0"
  sub a0, a0, a1            # a0 = delay - offset ; +1 bias
  jal Delay             # +1 bias

  # ~~~~ write return to 7SEG ~~~~
  sw s3, 0(s2)

  # ~~~~ syscall exit ~~~~
  li a7, 10     # load syscall num for exit()
  li a0, 0      # EXIT_SUCCESS code
  ecall       # execute sys call


# ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
# Purpose : delay(busy wait) of 0.5s
# Input :
#     a0 : [unsigned 32-bit] delay amount in instruction count
# Note :
#     - Consuming a0 in the process
# Return:
#

Delay:

  # ~~~~ calle(Delay) saves preserved ~~~~
  # ~~ 1. calle(Delay) allocates space on stack ~~
  addi sp, sp, -16              # +1 bias

  # ~~ 2. calle(Delay) stores reg values on stack ~~
  sw ra, 12(sp)               # +1 bias
  sw s1, 8(sp)                # +1 bias
  sw s2, 4(sp)                # +1 bias
  sw s3, 0(sp)                # +1 bias


```

```
81
82    # ~~ 3. calle(Delay) function execution ~~
83    # ~ while a0 <= 0
84  While: # loop = 3x instr
85    bleu a0, zero, EndWhile
86    addi a0, a0, -1
87    j While
88  EndWhile:
89
90    # ~~ 4. calle(Delay) restores preserved from stack ~~
91    lw ra, 12(sp)               # +1 bias
92    lw s1, 8(sp)                # +1 bias
93    lw s2, 4(sp)                # +1 bias
94    lw s3, 0(sp)                # +1 bias
95
96    # ~~ 5. calle(Delay) deallocates space on stack ~~
97    addi sp, sp, 16               # +1 bias
98
99    # ~~ calle(Delay) return ~~
100   jr ra                     # +1 bias
101 #~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

0.3 Appendix